

Week 13 - Routing and Templating Lab Report

Student Name: Enis Ögdüm

Date: January 12, 2026

Course: Dynamic Documents and Front-end Web Development

Introduction

This report documents the implementation of a Node.js Express application using the Twig template engine. The application demonstrates routing mechanisms and dynamic templating through three distinct pages, each handling different types of data processing.

1. Express Project Structure Overview

Project Generation with express-generator

This project was generated using the **express-generator** tool with the Twig template engine. The command used was:

```
npx express-generator --view=twig routing-templating-lab
```

This command creates a complete Express application structure with Twig as the templating engine.

Key Components and Their Purposes

1. app.js - The Main Application File

The **app.js** file is the **heart of the Express application**. It serves as the central configuration file where:

- **Express app is initialized:** Creates the main Express application instance
- **Middleware is configured:** Sets up body parsers, cookie parsers, static file serving, etc.
- **View engine is configured:** Specifies Twig as the template engine

```
app.set('views', path.join(__dirname, 'views'));  
app.set('view engine', 'twig');
```

- **Routes are registered:** Connects route modules to specific URL paths

```
app.use('/', indexRouter);  
app.use('/users', usersRouter);
```

- **Error handlers are defined:** Manages 404 errors and other application errors

2. routes/ Folder - Route Handlers

The **routes/** folder contains **route modules** that define the application's URL endpoints and their logic:

- **Purpose:** Separates routing logic from the main application file for better organization
- **Structure:** Each file typically handles a specific area of the application

- `index.js`: Handles routes for the home page and related pages
- `users.js`: Handles user-related routes (example)
- Custom route files can be added for specific features (e.g., `page1.js`, `page2.js`, `page3.js`)

Example Route Handler:

```
router.get('/page1', function(req, res, next) {
  const { a, b, c } = req.query;
  res.render('page1', { a, b, c });
});
```

3. views/ Folder - Twig Templates

The `views/` folder contains **Twig template files** (.twig) that define the HTML structure and presentation:

- **Purpose**: Separates presentation (HTML) from business logic (JavaScript)
- **Template Engine**: Twig allows dynamic content rendering using variables, conditionals, and loops
- **File Extension**: `.twig` files contain HTML with Twig syntax for dynamic content

Example Twig Template:

```
<!DOCTYPE html>
<html>
<head>
  <title>{{ title }}</title>
</head>
<body>
  <h1>{{ message }}</h1>
</body>
</html>
```

How Routing and Templating Are Connected

The connection between routing and templating follows this flow:

1. **Client Request** → User visits a URL (e.g., `http://localhost:3000/page1?a=hello&b=world&c=test`)
2. **Route Matching** → Express matches the URL to a route handler in the `routes/` folder

```
// In routes/index.js
router.get('/page1', function(req, res, next) { ... });
```

3. **Data Processing** → The route handler extracts parameters and prepares data

```
const { a, b, c } = req.query;
```

4. **Template Rendering** → The `res.render()` method connects routing to templating

```
res.render('page1', { a, b, c, title: 'Page 1' });
```

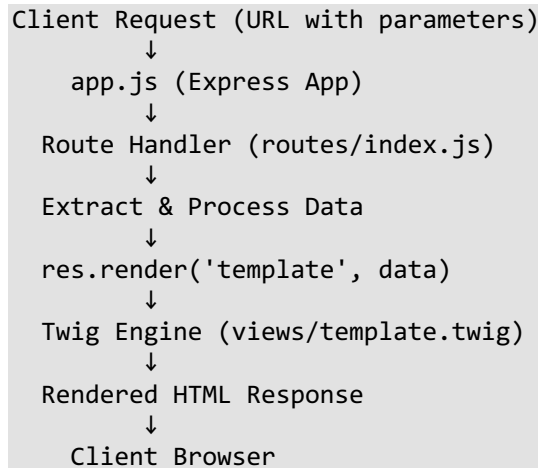
- **First parameter** (`'page1'`): Name of the template file (looks for `views/page1.twig`)
- **Second parameter** (`{ a, b, c }`): Data object passed to the template

5. **Template Processing** → Twig engine processes the template with the provided data

```
<p>Value A: {{ a }}</p>
<p>Value B: {{ b }}</p>
<p>Value C: {{ c }}</p>
```

6. **HTML Response** → The rendered HTML is sent back to the client's browser

Visual Flow Diagram



This architecture follows the **MVC (Model-View-Controller)** pattern where:

- **Routes** act as Controllers (handle requests and logic)
- **Views** are the Twig templates (presentation layer)
- **Models** would handle data (not implemented in this basic lab)

2. Main Project File - Routing Implementation

Route Handlers (routes/index.js)

Below are the route implementations added to the `routes/index.js` file:

Page 1 Route - Comment Detection (A, B, C Values)

```
router.get('/page1', function (req, res) {
  const value = req.query.value;
  let message;

  if (value === 'A') {
    message = 'Value A detected. This is option A.';
  } else if (value === 'B') {
    message = 'Value B detected. This is option B.';
  } else if (value === 'C') {
    message = 'Value C detected. This is option C.';
  } else {
    message = 'Unknown value provided.';
  }

  res.render('page1', { value, message });
});
```

Explanation:

- Accepts a **value** parameter from the URL query string (e.g., **?value=A**)
- Uses conditional logic to detect if the value is A, B, or C
- Generates an appropriate message based on the detected value
- Renders the **page1.twig** template with the value and message data

2. Page Implementations

Page 1: Value Detection (A, B, C Parameters)

Route: **/page1**

Method: GET

Parameters: **value** (accepts A, B, C, or other values)

Route Handler Explanation

The route handler in **routes/index.js** processes the **/page1** endpoint:

```
router.get('/page1', function (req, res) {
  const value = req.query.value;
  let message;

  if (value === 'A') {
    message = 'Value A detected. This is option A.';
  } else if (value === 'B') {
    message = 'Value B detected. This is option B.';
  } else if (value === 'C') {
    message = 'Value C detected. This is option C.';
  } else {
    message = 'Unknown value provided.';
  }

  res.render('page1', { value, message });
});
```

How it works:

- The route reads the URL parameter **value** from **req.query.value**
- It checks whether the value is A, B, C, or something else using conditional statements
- Based on the value, it sets a corresponding message:
 - o **A** → "Value A detected. This is option A."
 - o **B** → "Value B detected. This is option B."
 - o **C** → "Value C detected. This is option C."
 - o **Any other value** → "Unknown value provided."

- The route then renders the `page1.twig` template, passing `value` and `message` as variables

Twig Template Code

The template file `views/page1.twig` displays the dynamic content:

```
<!DOCTYPE html>
<html>
<head>
  <title>Page 1 - Value Detection</title>
</head>
<body>
  <h1>Page 1 - Value Detection</h1>
  <p>Provided value: {{ value }}</p>
  <p>{{ message }}</p>
</body>
</html>
```

Template features:

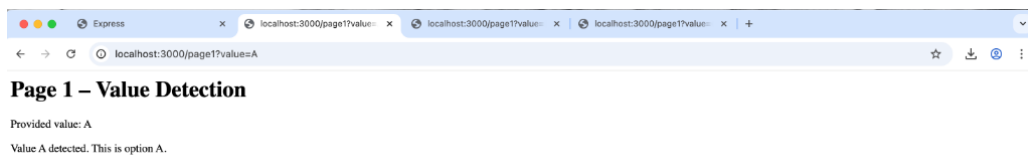
- Displays a heading "Page 1 – Value Detection"
- Shows the provided value using `{{ value }}`
- Displays the message dynamically using `{{ message }}`
- Twig's double curly braces `{{ }}` are used to output variables passed from the route handler

How Route and Template Work Together

1. **User Request:** When a user opens `/page1?value=A` (or B, C, etc.) in their browser
2. **Route Execution:** Express matches the URL to the `/page1` route handler
3. **Data Processing:** The route extracts the `value` parameter and determines the appropriate message
4. **Template Rendering:** The `res.render()` method passes the data to `page1.twig`
5. **Dynamic Display:** The Twig template receives the variables and renders them as HTML
6. **Browser Output:** The user sees the heading, their provided value, and the corresponding message

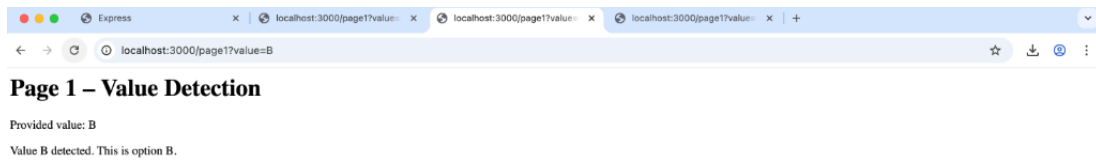
Screenshots

Screenshot 1: Page 1 with value=A



- . **URL**
shown: `http://localhost:3000/page1?value=A`
- **Content displayed:**
 - Heading: "Page 1 – Value Detection"
 - Provided value: A
 - Message: "Value A detected. This is option A."

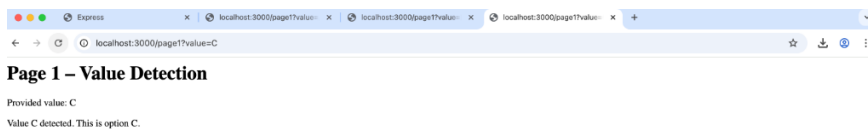
Screenshot 2: Page 1 with value=B



- .
- **shown:** `http://localhost:3000/page1?value=B`
- **Content displayed:**
 - o Heading: "Page 1 – Value Detection"
 - o Provided value: B
 - o Message: "Value B detected. This is option B."



Screenshot 3: Page 1 with value=C



-
-

- **URL shown:** `http://localhost:3000/page1?value=C`
- **Content displayed:**
 - o Heading: "Page 1 – Value Detection"
 - o Provided value: C
 - o Message: "Value C detected. This is option C."

Note: Similar behavior occurs for unknown values, displaying "Unknown value provided."

Page 2: Numeric Value Detection

Route: `/page2`

Method: GET

Parameters: `number` (numeric value or invalid input)

Route Handler Explanation

The route handler in `routes/index.js` processes the `/page2` endpoint:

```
router.get('/page2', function (req, res) {
  const number = Number(req.query.number);
  let message;

  if (isNaN(number)) {
    message = 'Invalid number provided.';
  } else if (number > 0) {
    message = 'The number is positive.';
  } else if (number < 0) {
    message = 'The number is negative.';
  } else {
    message = 'The number is zero.';
  }

  res.render('page2', { number: req.query.number, message });
});
```

How it works:

- The route reads the URL parameter `number` from `req.query.number`
- It converts the value to a number using `Number()` and checks for invalid input
- Based on the numeric value, it sets a corresponding message:
 - o **Positive number** → "The number is positive."
 - o **Negative number** → "The number is negative."
 - o **Zero** → "The number is zero."
 - o **Not a number** → "Invalid number provided."
- The route renders the `page2.twig` template, passing `number` and `message` as variables

Twig Template Code

The template file `views/page2.twig` displays the dynamic content:

```
<!DOCTYPE html>
<html>
<head>
  <title>Page 2 - Number Analysis</title>
</head>
<body>
  <h1>Page 2 - Number Analysis</h1>
  <p>Provided number: {{ number }}</p>
  <p>{{ message }}</p>

  <hr>
  <p>Try URLs like:</p>
  <ul>
    <li>/page2?number=10</li>
    <li>/page2?number=-5</li>
    <li>/page2?number=0</li>
    <li>/page2?number=abc</li>
  </ul>
</body>
</html>
```

Template features:

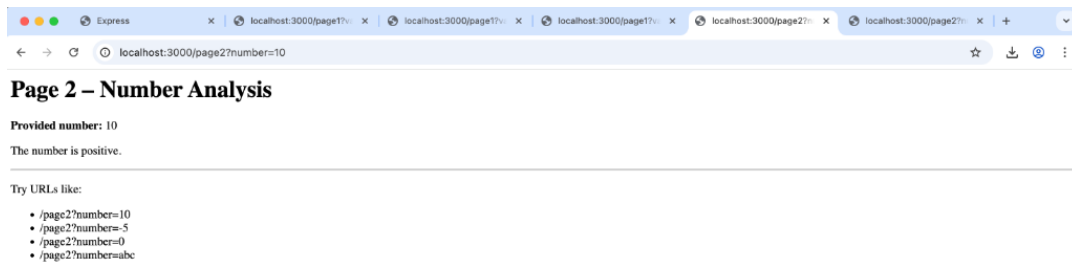
- Displays a heading "Page 2 – Number Analysis"
- Shows the provided number using `{{ number }}`
- Displays the analysis message dynamically using `{{ message }}`
- Includes helpful examples of URLs to try

How Route and Template Work Together

1. **User Request:** User opens `/page2?number=10` (or any other value) in their browser
2. **Route Execution:** Express matches the URL to the `/page2` route handler
3. **Data Processing:** The route converts the parameter to a number and determines if it's positive, negative, zero, or invalid
4. **Template Rendering:** The `res.render()` method passes the number and message to `page2.twig`
5. **Dynamic Display:** The Twig template receives the variables and renders them as HTML
6. **Browser Output:** The user sees the number and the corresponding analysis message

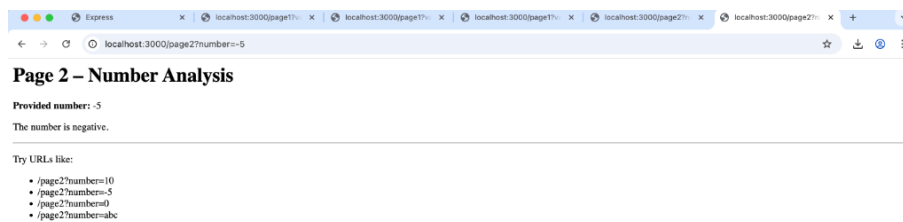
Screenshots

Screenshot 1: Page 2 with number=10 (Positive)



- URL shown: **http://localhost:3000/page2?number=10**
- Content displayed:
 - Heading: "Page 2 – Number Analysis"
 - Provided number: 10
 - Message: "The number is positive."

Screenshot 2: Page 2 with number=-5 (Negative)



- URL shown: **http://localhost:3000/page2?number=-5**
- Content displayed:

- Heading: "Page 2 – Number Analysis"
- Provided number: -5
- Message: "The number is negative."

Page 3: Bullet Point List Generator

Page 3: List Rendering

Route: `/page3`

Method: GET

Parameters: `items` (comma-separated list of strings)

Route Handler Explanation

The route handler in `routes/index.js` processes the `/page3` endpoint:

```
router.get('/page3', function (req, res) {
  const items = req.query.items ? req.query.items.split(',') : [];

  // Clean up the items (trim whitespace and filter empty strings)
  const cleanItems = items
    .map(item => item.trim())
    .filter(item => item.length > 0);

  res.render('page3', { items: cleanItems });
});
```

How it works:

- The route reads the URL parameter `items` from `req.query.items`
- It splits the comma-separated string into an array using `.split(',')`
- It processes the array to:
 - Trim whitespace from each item
 - Remove any empty items
- The route passes the `cleanItems` array to the `page3.twig` template

Twig Template Code

The template file `views/page3.twig` generates the list dynamically:

```
<!DOCTYPE html>
<html>
<head>
  <title>Page 3 - List Rendering</title>
</head>
```

```

<body>
  <h1>Page 3 - List Rendering</h1>

  <p>Provided items:</p>
  <ul>
    {% for item in items %}
      <li>{{ item }}</li>
    {% else %}
      <li>No items provided.</li>
    {% endfor %}
  </ul>

  <hr>
  <p>Try URL like:</p>
  <ul>
    <li>/page3?items=apple,banana,orange</li>
    <li>/page3?items=dog,cat,mouse</li>
  </ul>
</body>
</html>

```

Template features:

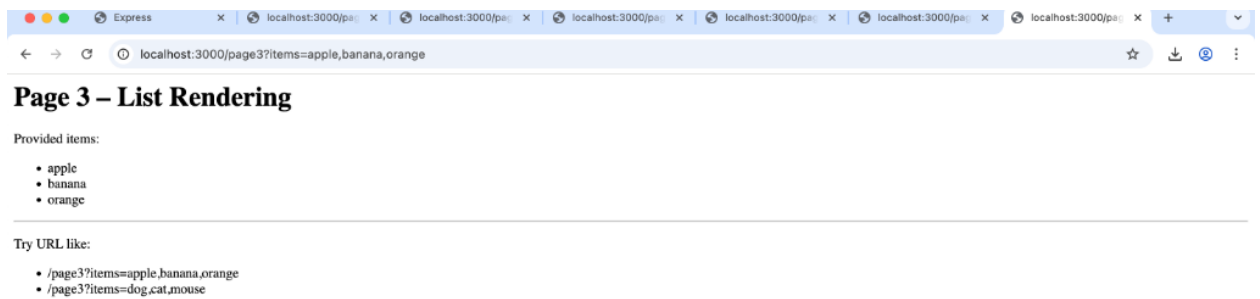
- Displays a heading "Page 3 – List Rendering"
- Uses a `{% for %}` loop to iterate through the `items` array
- Generates a `<li>` bullet point for each item in the list
- Includes an `{% else %}` block to handle cases where no items are provided

How Route and Template Work Together

1. **User Request:** User opens `/page3?items=apple,banana,orange`
2. **Route Execution:** Express converts the string "apple,banana,orange" into `["apple", "banana", "orange"]`
3. **Template Rendering:** The array is passed to the view
4. **Loop Execution:** The Twig template iterates over the array and generates HTML `<li>` elements for each item
5. **Browser Output:** The user sees a formatted bullet point list

Screenshots

Screenshot 1: List Generation



– **URL shown:** `http://localhost:3000/page3?items=apple,banana,orange`

– **Content displayed:**

- Heading: "Page 3 – List Rendering"
- Bulleted list:
 - apple
 - banana
 - orange

3. Conclusion

This lab successfully demonstrates:

- Express routing with GET parameters
- Twig template engine integration
- Dynamic content rendering based on URL parameters
- Conditional logic in templates (Page 1 & 2)
- Loop iteration for list generation (Page 3)

References

- Express Generator Documentation: <https://expressjs.com/en/starter/generator.html>
- Twig.js Documentation: <https://github.com/twigjs/twig.js>
- Node.js Documentation: <https://nodejs.org/>

End of Report